



Landmarks (landmarks)

La ville de Luxembourg est coupée en deux par la vallée de l'Alzette. Lea et Tom se promènent de chaque côté de la vallée, visitant différents monuments. Le long du chemin de Lea, il y a N monuments, et le long de celui de Tom, il y en a M .

Les monuments sont de trois types :

1. une ancienne fortification,
2. une statue, ou
3. une fontaine.

Les monuments sur le chemin de Lea sont $A_0, A_1, \dots, A_{N-1}$, et ceux sur le chemin de Tom sont $B_0, B_1, \dots, B_{M-1}$, chaque valeur étant 1, 2 ou 3.

Ils veulent tous deux découvrir la ville ensemble, mais veulent s'assurer d'être à tout moment devant le même type de monument.

Considérons un quadruplet d'entiers (X_1, X_2, Y_1, Y_2) , décrivant les deux segments qu'ils prévoient de parcourir, satisfaisant :

- $0 \leq X_1 \leq X_2 < N$ – représentant le segment de Lea commençant à X_1 et se terminant à X_2 ,
- $0 \leq Y_1 \leq Y_2 < M$ – représentant le segment de Tom commençant à Y_1 et se terminant à Y_2 ,
- $A_{X_1} = B_{Y_1}$ – ils commencent par le même type de monument, et
- $A_{X_2} = B_{Y_2}$ – ils finissent par le même type de monument.

Ils commencent tous deux au début de leurs segments respectifs : Lea commence à $X = X_1$, et Tom à $Y = Y_1$.

À chaque étape, Lea choisit $x \in \{-1, 0, 1\}$, et Tom choisit $y \in \{-1, 0, 1\}$, ce qui correspond à aller au monument précédent, au suivant, ou à rester sur place. Ils mettent ensuite à jour leurs positions $X + x$ et $Y + y$ à condition qu'après l'étape, ils vérifient :

- $X_1 \leq X \leq X_2$ – Lea reste dans son segment,
- $Y_1 \leq Y \leq Y_2$ – Tom reste dans son segment, et
- $A_X = B_Y$ – ils se trouvent devant le même type de monument.

Le quadruplet (X_1, X_2, Y_1, Y_2) est dit valide s'il est possible d'appliquer zéro ou plusieurs étapes jusqu'à ce que $X = X_2$ et $Y = Y_2$.

On vous donne Q requêtes. Dans chaque requête, on vous donne un quadruplet de quatre entiers (X_1, X_2, Y_1, Y_2) et votre tâche est de déterminer si le quadruplet est valide.

Implémentation

Vous devrez soumettre un unique fichier source `.cpp`.



Parmi les pièces jointes de cette tâche, vous trouverez un modèle `landmarks.cpp` avec un squelette d'implémentation.

Vous devrez implémenter les fonctions suivantes :

C++

```
void init(vector<int> A, vector<int> B)
```

- A est un tableau de N entiers.
- B est un tableau de M entiers.
- Cette procédure est appelée exactement une fois, avant tout appel à `query`.

C++	<code>bool query(int X1, int X2, int Y1, int Y2)</code>
-----	---

- X_1, X_2, Y_1 et Y_2 sont des entiers satisfaisant les conditions suivantes :
 - $0 \leq X_1 \leq X_2 < N$,
 - $0 \leq Y_1 \leq Y_2 < M$,
 - $A_{X_1} = B_{Y_1}$, et
 - $A_{X_2} = B_{Y_2}$.
- Si le tuple (X_1, X_2, Y_1, Y_2) est valide, la fonction doit retourner `true`. Sinon, elle doit retourner `false`.
- Cette procédure est appelée exactement Q fois.

Évaluateur

Parmi les pièces jointes de ce problème, vous trouverez une version simplifiée de l'évaluateur (`grader`) utilisé lors de l'évaluation, que vous pouvez utiliser pour tester vos solutions localement. L'évaluateur d'exemple lit les données depuis `stdin`, appelle la fonction `init` une fois, appelle la fonction `query` Q fois et écrit en retour sur `stdout` en utilisant le format suivant.

L'entrée est composée de $Q + 3$ lignes, contenant :

- Ligne 1 : les entiers N, M et Q .
- Ligne 2 : N entiers, $A_0, A_1, \dots, A_{N-1}$.
- Ligne 3 : M entiers, $B_0, B_1, \dots, B_{M-1}$.
- Ligne $4 + i$ ($0 \leq i < Q$) : les entiers X_1, X_2, Y_1 et Y_2 .

La sortie est composée de Q lignes, où la i -ème ligne contient la valeur retournée par le i -ème appel à la fonction `query` (`true` ou `false`).

Contraintes

- $1 \leq N, M, Q \leq 100\,000$.
- $1 \leq A_i, B_i \leq 3$ pour tout i .

Score

Votre programme sera testé sur plusieurs tests, regroupés en sous-tâches. Pour obtenir les points d'une sous-tâche, votre programme doit résoudre correctement tous les tests qui lui sont associés.

- **Sous-tâche 0 [0 points]**: Exemples.
- **Sous-tâche 1 [10 points]**: $N, M, Q \leq 5000$ et $A_0, A_1, \dots, A_{N-1}, B_0, B_1, \dots, B_{M-1} \leq 2$. (il n'y a que deux types de monuments)
- **Sous-tâche 2 [10 points]**: $A_0, A_1, \dots, A_{N-1}, B_0, B_1, \dots, B_{M-1} \leq 2$. (il n'y a que deux types de monuments)
- **Sous-tâche 3 [10 points]**: $N, M, Q \leq 5000$ et $A_i = (i \bmod 3) + 1$ pour tout i .
- **Sous-tâche 4 [15 points]**: $A_i = (i \bmod 3) + 1$ pour tout i .
- **Sous-tâche 5 [10 points]**: $N, M, Q \leq 300$.
- **Sous-tâche 6 [25 points]**: $N, M, Q \leq 5000$.
- **Sous-tâche 7 [20 points]**: Aucune contrainte supplémentaire.

Exemples

stdin	stdout
5 7 4	true
1 2 3 2 1	false
1 1 2 3 2 3 1	true
0 2 0 5	false
2 4 3 6	
1 1 4 4	
1 1 2 4	

Explication

$N = 5$, $M = 7$, $Q = 4$, $A = [1, 2, 3, 2, 1]$ et $B = [1, 1, 2, 3, 2, 3, 1]$.

Requête 1: $(X_1, X_2, Y_1, Y_2) = (0, 2, 0, 5)$ est valide car le parcours suivant est possible :

étape	1	2	3	4	5	6
X	0	0	1	2	1	2
Y	0	1	2	3	4	5
type de monument	1	1	2	3	2	3

Requête 2: $(X_1, X_2, Y_1, Y_2) = (2, 4, 3, 6)$ n'est pas valide. Aucun parcours ne satisfait les conditions.

Requête 3: $(X_1, X_2, Y_1, Y_2) = (1, 1, 4, 4)$ est valide.

Requête 4: $(X_1, X_2, Y_1, Y_2) = (1, 1, 2, 4)$ n'est pas valide.